

Romanian legal document simplification pipeline ▾



Role: You are an expert in AI Agentic pipelines for Humanist projects who wants to develop my pipeline for my project.

Task: Create a pipeline using OpenAI Agents SDK (see attached notebook for inspiration) in a Jupyter Notebook for simplifying Romanian legal documents to plain language based on examples.

My Idea: Load the legal documents as input from a folder called "data". First have an AI Agent simplify the language to plain (Romanian) language. The criteria for simplification should be provided to the Agent as a markdown file. Secondly, using ReaderBench, evaluate the readability of the original, raw document and then compare the score to that of the simplified version using objective measures (see attachment ICTAI-2016.PDF part III for indices). Third, use a second AI Agent to output a new PDF with the name of the file + "_simplified" in the folder "simplified_docs" which includes: A summary of the most important points of the document, the ReaderBench score comparison and the plain language version created by the first agent.

Special requests:

- pay close attention to the provided markdown file
- after ingesting all the information, ask if anything is unclear
- provide a very brief assessment of the idea and what could be added/improved
- Wait for my go before generating the notebook

Show less

Assessed project scope and deferred notebook generation pending clarification

I'll start by reading the uploaded notebook to understand the SDK patterns you want me to follow. The ICTAI paper and CLAUDE.md are already in my context.

Assessed technical feasibility and identified critical clarifications needed

I've read all three files. Quick confirmation of what I'm working from: the notebook establishes your SDK idioms (`Agent` , `Runner.run` `async` , `agent.as_tool(...)` , `handoff` , `InputGuardrail` , `trace` , `DEFAULT_MODEL = 'gpt-5.4-mini'` , `.txt` loaded from `data/` , `.env` for the key); the ICTAI paper's Section III gives the ReaderBench index families; and CLAUDE.md sets the working style (surface assumptions, stay minimal, ask before building). Per your instructions I have **not** generated anything yet.

Brief assessment of the idea

The three-stage shape (simplify → measure → package) is sound and maps cleanly onto the SDK. Two agents plus two deterministic steps (ReaderBench scoring, PDF writing) is the right division of labor — you don't want an LLM *rendering* PDFs or *computing* metrics, only producing prose.

Three things worth knowing before we commit:

1. **"ReaderBench score" isn't a single number.** ReaderBench (and the ICTAI paper) produce a *vector* of indices, and the paper uses them to *differentiate* styles via MANOVA, not to emit one readability figure. For a before/after comparison I'd recommend a **table of indices with a direction-of-simplification column** (e.g., shorter words ↓, shorter sentences ↓, lower word entropy ↓, higher cohesion ↑ = simpler) rather than inventing a composite "score." Flesch-style formulas exist but aren't validated for Romanian, so I'd avoid them.
2. **The cohesion indices are the expensive half.** The lexical indices from §III.B.1 (word length, unique content words/sentence, word-stem entropy, lemma↔stem distances) are cheap — they need only a Romanian spaCy model. The cohesion/semantic indices from §III.B.2 (paragraph-document, sentence-paragraph, intra-paragraph, transition cohesion) need **trained Romanian LSA/LDA models**, which are the awkward part of a ReaderBench install. We should decide whether to compute the full set or a robust lexical subset.
3. **Romanian diacritics in the PDF** (ă â î ș ț) will silently break with default PDF-library fonts. Whatever writer we use needs a Unicode font (e.g., DejaVu) registered. Minor, but a classic failure I want to lock down.

Worth adding later (not now, unless you want it): a fidelity guardrail — legal simplification can quietly drop meaning, so a check that the plain version didn't lose obligations/deadlines/parties would be genuinely valuable for this domain, and a one-line "not legal advice" disclaimer on the PDF.

What I need clarified before writing anything

1. **The simplification-criteria markdown.** You said to "pay close attention to the provided markdown file" and that criteria will be "provided to the Agent as a markdown file" — but the only markdown in the uploads is `CLAUDE.md` (the behavioral guide). Did you mean I should follow `CLAUDE.md` as the working contract, **and** the criteria file is still to come? If so, can you share it (or tell me its intended filename/path and rough structure), since it drives Agent 1 entirely?
2. **Input format.** The example loads `.txt`, but "legal documents" + PDF output suggests the inputs might be `.pdf` or `.docx`. What's actually in `data/`? If PDFs, I'll add a text-extraction step (pdfplumber).
3. **"Based on examples."** What are the examples — few-shot before/after simplification pairs? Are they inside the criteria markdown, or a separate set I should load?
4. **ReaderBench availability.** Do you already have `readerbench` installed with the Romanian models, and do you want the **full** index set (incl. LSA/LDA cohesion) or the **lexical subset**? Note: this environment has **no network**, so I can write the code against the documented API but I won't be able to *run* the ReaderBench portion here — install/model verification will happen on your machine.
5. **Scaffolding scope.** Your flow is linear; the example's guardrail + human-review machinery isn't obviously needed. I plan to drop it and keep two agents + two tools (score, write-PDF). Object if you want the guardrail kept.

Give me your answers and your go, and I'll build the notebook.

1. Yes, I meant `CLAUDE.md`
2. It's PDFs
3. I meant the principles of plain language, but adapted to Romanian:
<https://digital.gov/guides/plain-language/principles#content-start> ;
<https://digital.gov/guides/plain-language/principles/short-simple#content-start> ; <https://digital.gov/guides/plain-language/principles/avoid-jargon#content-start>
4. I do not have it installed. Please provide indications on how to install it.
 What is the difference between the full index and lexical subset for the workflow?
5. I agree.

Orchestrated resource fetching and installation research across multiple domains

Good — let me pull the plain-language principles you linked (they'll seed the criteria markdown) and check current ReaderBench install specifics so my guidance is accurate rather than from memory.

Fetches: Principles of plain language

I have the overview. Let me grab the two specific sub-pages you linked.

Assembling installation guidance and indexing distinctions

I have the plain-language material. Now let me verify current ReaderBench install details.

Weighed installation complexity and index scope for Romanian text analysis

Got everything I need. Here's where things stand.

Plain-language criteria file

I pulled the three digital.gov pages. I'll translate their operative rules into a Romanian-adapted `criteria.md` that Agent 1 loads at runtime — covering short sentences and cutting filler, active voice (*diateza activă*), avoiding jargon while keeping necessary legal terms (with a define-in-place rule), removing doublets/triplets (very common in Romanian legalese, e.g. "*de îndată și fără întârziere*"), and term consistency. I'll keep it as an editable markdown so you can tune it without touching code.

ReaderBench install — and an important recommendation

The official Python package is `rbpy-rb` (imports as `rb`). Realistically, you should know upfront: it's installed via `pip3 install --user rbpy-rb`, then you download the spaCy models yourself with `python -m spacy download <model>` (Romanian: `ro_core_news_lg`), run its `nltk_download.py`, and the trained semantic models live on HuggingFace at huggingface.co/readerbench. The catch: the package was last meaningfully published around 2019 and pins older spaCy/Python versions, so it's fragile on modern Python (3.11+). It usually needs a dedicated Python 3.8 virtualenv to install cleanly. `PyPI + 2`

This connects directly to your question about **full vs. lexical**, because the two index families have very different install costs:

Lexical subset (ICTAI §III.B.1 — word length, unique content words/sentence, word-stem entropy, lemma↔stem distances). These measure word- and sentence-level simplicity — exactly what plain-language editing changes most. I can compute these **directly from the Romanian spaCy model** (`ro_core_news_lg`), no `rbpy-rb`, no trained semantic models. Modern, robust, testable. And the paper itself found style differences were most strongly

reflected in lexical rather than cohesion properties — so this is where your before/after signal will be clearest. `digital` `digital`

Full set adds cohesion/semantics (§III.B.2 — paragraph-document, sentence-paragraph, intra-paragraph, transition cohesion via LSA/LDA). These need ReaderBench's trained Romanian LSA/LDA models and thus the fragile `rbpy-rb` stack. Cohesion is also an ambiguous simplification signal — splitting a dense sentence can *lower* measured cohesion even though the text got easier.

My recommendation: build the pipeline on the lexical subset via spaCy (robust, directly relevant, installs in one line: `pip install spacy + python -m spacy download ro_core_news_lg`), and structure the scoring function so the LSA/LDA cohesion indices can be slotted in later *if* you get `rbpy-rb` running. You get a working, reproducible readability comparison without the 2019-era dependency hell.

Two remaining flags (both handled, just noting)

- **PDF diacritics:** I'll register a Unicode font (DejaVuSans) so ă/â/î/ș/ț render correctly — I'll include a download note since this env is offline.
- **Legal fidelity:** the output PDF will carry a short "orientational, not legal advice" disclaimer, since plain-language rewriting can shift legal meaning.

One decision left, then I'll build:

Go with the spaCy-based lexical subset as the default (recommended), or do you want me to wire up the full `rbpy-rb` cohesion stack despite the install fragility? Confirm that plus a go, and I'll generate the notebook.

go with the spaCy lexical subset and generate the notebook, please!

Architected comprehensive legal document simplification pipeline with dual agents

Go it is. Let me build the criteria file first, then the notebook.

Validating notebook builds and sanity-checking Python logic

Now the notebook. I'll assemble it with a builder script for reliability.

Deliberated testing strategy, opted for pure logic validation

Notebook is valid and every code cell parses. Let me sanity-check the pure-Python logic (edit distance, entropy, comparison) offline so I'm not shipping a silent bug in the scoring.

Unit-test the deterministic scoring helpers

Logic checks pass. Both files are ready.

Presented 2 files

Done. Two files: the notebook and the `criteria.md` that Agent 1 loads.

How it's wired

- Loads PDFs from `data/` (pdfplumber), keeping the SDK idioms from your reference notebook: `Agent`, `async`, `Runner.run`, `trace`, `DEFAULT_MODEL='gpt-5.4-mini'`, `.env`. No guardrail/handoff scaffolding, as agreed.
- **Agent 1** rewrites to plain Romanian using `criteria.md` (injected into its instructions).
- **Readability** computes the five ICTAI §III.B.1 lexical indices with `ro_core_news_lg` on original vs. simplified, with a direction column (lower = simpler).
- **Agent 2** produces the key-points summary; a deterministic writer builds `simplified_docs/<name>_simplified.pdf` with summary + comparison table + plain-language version + a "not legal advice" disclaimer, using a DejaVu font so diacritics render.

Before running

```
bash
```

```
pip install openai-agents pdfplumber spacy fpdf2 python-dotenv matplotlib
python -m spacy download ro_core_news_lg
```

Put `criteria.md` and a `.env` (with `OPENAI_API_KEY`) next to the notebook, and your PDFs in `data/`.

Two honesty flags baked into the notebook (both in markdown cells so they're visible to you and any collaborators): spaCy lemmas stand in for the paper's Romanian "stems," so I collapsed its two stem-distance indices into one word-lemma edit distance; and the cohesion/semantic indices are deliberately excluded, with the scoring function shaped so you can slot them in later if you get the trained Romanian LSA/LDA models running.

I verified the JSON, that every code cell parses, and unit-tested the scoring helpers offline — but I couldn't execute the spaCy/agent/PDF path here since this environment has no network and no `ro_core_news_lg`. The first real run on your machine is the true test; the

One thing worth considering as a next step: a fidelity check comparing the simplified text back against the original for dropped obligations or deadlines — genuinely useful for legal text, but I left it out since you didn't ask for it.

Show more

What the error means. `FPDFException: Not enough horizontal space to render a single character` is `fpdf2`'s way of saying "I was asked to lay out text in a box whose usable width is essentially zero." A `multi_cell(0, ...)` means "use the full width from the current cursor position to the right margin." If the cursor is already sitting at (or very near) the right margin when that call runs, the available width is ~0, and even one character won't fit — hence the exception.

Why the cursor is at the right margin. In `fpdf2`, `multi_cell` moves the cursor *down* to the next line but leaves the horizontal position at the *right edge* of what it just drew, not back at the left margin. It doesn't auto-return to the left. In the notebook I mostly get away with this because nearly every `multi_cell` is followed by an `ln(...)` call, and `ln()` happens to reset `x` back to the left margin — so it silently papers over the problem.

The one place that pattern breaks is exactly where it crashed:

```
pdf.multi_cell(0, 8, 'Rezumat - puncte importante') # heading - leaves x at
right margin
pdf.set_font('DejaVu', '', 11)
pdf.multi_cell(0, 6, summary) # starts at right margin
→ width ≈ 0 → boom
```

There's no `ln()` between the heading and the body, so the summary call inherits the cursor at the right margin. Note the traceback lands right here (the summary `multi_cell`), which matches this diagnosis exactly.

Two consequences worth knowing:

- The same latent bug exists further down for the "*Versiune in limbaj simplu*" heading immediately followed by the `simplified` body — you'd hit it there next once the summary is fixed.
- The table is *not* affected, because it uses `cell()` with manual `set_xy()`, so I'm controlling `x` there explicitly.

So the fix is a one-liner in spirit: make each `multi_cell` return the cursor to the left margin — either by passing an explicit `new_x` argument, or by resetting `x` (e.g. a bare `ln()` / `set_x`) between the heading and its body in the two spots that lack it. Nothing about the scoring or the agents needs to change.

Want me to apply that fix? (Separately, and unrelated to the crash: two of your five indices came out "mai complex" — word length and word-lemma distance — which is a real, expected artifact of how those are measured on legal text, not a bug. Happy to explain that too if useful.)

Please generate only the section / cell where the change has to be made (and not the entire notebook). Yes, provide a brief (5-7 sentence explanation for the "mai complex" result).

Diagnosed fpdf2 cursor positioning bug and engineered targeted fix

Here's the corrected Step 5 cell. The only substantive change is importing `XPos`, `YPos` and adding `new_x=XPos.LMARGIN`, `new_y=YPos.NEXT` to every full-width `multi_cell`, which forces the cursor back to the left margin after each block so the next one gets the full page width. The table is untouched, since it already controls position manually with `set_xy`.

python

```
from fpdf import FPDF
from fpdf.enums import XPos, YPos
```

```
def _find_font(filename: str):
    search = []
    try:
        import matplotlib
        search.append(Path(matplotlib.get_data_path()) / 'fonts' / 'ttf')
    except Exception:
        pass
    search += [Path('/usr/share/fonts/truetype/dejavu'),
               Path('/Library/Fonts'), Path('C:/Windows/Fonts')]
    for d in search:
        p = d / filename
        if p.exists():
            return str(p)
    return None
```

```
REGULAR_TTF = _find_font('DejaVuSans.ttf')
BOLD_TTF = _find_font('DejaVuSans-Bold.ttf') or REGULAR_TTF
assert REGULAR_TTF, ('DejaVuSans.ttf not found. Install matplotlib (it bundles the
                     'REGULAR_TTF / BOLD_TTF to a Unicode .ttf so Romanian diacriti
```

```
DISCLAIMER = ('Acest document este o versiune orientativa in limbaj simplu, generat
              'Nu constituie consultanta juridica si nu inlocuieste textul oficial.
```

```
# Full-width text blocks return the cursor to the left margin after each call.
FULL = dict(new_x=XPos.LMARGIN, new_y=YPos.NEXT)
```

```

def build_pdf(out_path: Path, title: str, summary: str, rows: list[dict], simplified: bool):
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_font('DejaVu', '', REGULAR_TTF)
    pdf.add_font('DejaVu', 'B', BOLD_TTF)
    pdf.add_page()

    pdf.set_font('DejaVu', 'B', 15)
    pdf.multi_cell(0, 9, title, **FULL)
    pdf.ln(2)

    pdf.set_font('DejaVu', '', 9)
    pdf.set_text_color(90, 90, 90)
    pdf.multi_cell(0, 5, DISCLAIMER, **FULL)
    pdf.set_text_color(0, 0, 0)
    pdf.ln(4)

    pdf.set_font('DejaVu', 'B', 12)
    pdf.multi_cell(0, 8, 'Rezumat - puncte importante', **FULL)
    pdf.set_font('DejaVu', '', 11)
    pdf.multi_cell(0, 6, summary, **FULL)
    pdf.ln(4)

    pdf.set_font('DejaVu', 'B', 12)
    pdf.multi_cell(0, 8, 'Comparatie lizibilitate (indici lexicali ReaderBench)', **FULL)
    pdf.ln(1)

    headers = ['Indice', 'Original', 'Simplificat', 'Delta', 'Evaluare']
    widths = [72, 24, 26, 20, 30]
    pdf.set_font('DejaVu', 'B', 9)
    for h, w in zip(headers, widths):
        pdf.cell(w, 7, h, border=1)
    pdf.ln(7)

    pdf.set_font('DejaVu', '', 9)
    for row in rows:
        values = [row['index'], f"{row['original']}", f"{row['simplified']}",
                  f"{row['delta']:+.3f}", row['evaluare']]
        x0, y0 = pdf.get_x(), pdf.get_y()
        pdf.multi_cell(widths[0], 7, values[0], border=1)
        y1 = pdf.get_y()
        h = y1 - y0
        pdf.set_xy(x0 + widths[0], y0)

```

```
for v, w in zip(values[1:], widths[1:]):
    pdf.cell(w, h, v, border=1)
pdf.set_xy(x0, y1)

pdf.ln(2)
pdf.set_font('DejaVu', '', 8)
pdf.set_text_color(90, 90, 90)
pdf.multi_cell(0, 4, 'Nota: valori mai mici indica, in general, un text mai sin
pdf.set_text_color(0, 0, 0)
pdf.ln(4)

pdf.set_font('DejaVu', 'B', 12)
pdf.multi_cell(0, 8, 'Versiune in limbaj simplu', **FULL)
pdf.set_font('DejaVu', '', 11)
pdf.multi_cell(0, 6, simplified_text, **FULL)

pdf.output(str(out_path))
return out_path
```

Replace your current Step 5 cell with this, re-run it, then re-run the final cell.

On the two "mai complex" results. First, the word-length change is 6.0 → 6.069 — about a 1% difference, which is essentially noise rather than a real regression. Both flagged indices are averages computed *only over content words*, so they're driven by which words remain after plain-language editing, not by the text getting harder. Simplification strips out lots of short function words and can swap terse legalese and archaic particles for ordinary content words, which tends to nudge the average content-word length slightly upward even as the text reads more easily. The word-lemma distance (inflection proxy) rises for a related reason: everyday Romanian verbs and nouns are often more heavily inflected than the fixed, formulaic legal boilerplate they replace, so the surface-to-lemma edit distance grows. It's also the shakiest of the five indices, since it leans on the lemmatizer's behavior. The signals that actually track readability here — sentence length, unique words per sentence, and entropy — all moved clearly toward simpler, which is the outcome you want.

